TypedArray / ArrayBuffer / SharedArrayBuffer Concatenation

Proposing advancement to **Stage 2**

James M Snell

Recap: The Problem

Concatenating TypedArrays and ArrayBuffers is a common operation with no good built-in solution.

```
// The current approach — manual allocation and copying
function concat(buffers, size) {
  const dest = new Uint8Array(size);
  let offset = 0;
  for (const buffer of buffers) {
    dest.set(buffer, offset);
    offset += buffer.length;
  }
  return dest;
}
```

- Requires user-managed allocation, offset tracking, and copying
- `TypedArray.prototype.set` works one item at a time — no batch operation
- No affordance for implementation-defined optimization (bulk copy, deferred alloc, etc.)
- Common pattern in streams, networking, file I/O — browser and server

The Solution: Three Complementary Methods

Method	Inputs	Returns
`%TypedArray%.concat(items [, length])`	Same-type TypedArrays	TypedArray
`ArrayBuffer.concat(items [, options])`	AB, SAB, TA, DataView	ArrayBuffer
`SharedArrayBuffer.concat(items [, options])`	AB, SAB, TA, DataView	SharedArrayBuffer

Why three?

- `%TypedArray%.concat` — **element-oriented**, the right level for typed data
- `ArrayBuffer.concat` / `SharedArrayBuffer.concat` — **byte-oriented**, the right level for buffer properties (resizable, immutable, growable)
- Mirrors existing `ArrayBuffer` / `SharedArrayBuffer` constructor separation

`%TypedArray%.concat(items [, length])`

Static method on each TypedArray constructor. Concatenates same-type TypedArrays.

```
const u8_1 = new TextEncoder().encode('Hello ');
const u8_2 = new TextEncoder().encode('World!');

const result = Uint8Array.concat([u8_1, u8_2]);
// Uint8Array [72, 101, 108, 108, 111, 32, 87, 111, 114, 108, 100, 33]
```

Key behaviors:

- `items` — iterable of TypedArrays, **all must be the same type** as the constructor
- `length` (optional) — truncate or zero-fill the result
- Items may be backed by ArrayBuffer **or** SharedArrayBuffer
- Throws `TypeError` for mismatched types or detached buffers
- Available on all constructors: `Int8Array`, `Uint8Array`, `Float64Array`, etc.

`ArrayBuffer.concat(items [, options])`

Byte-level concatenation into a new ArrayBuffer. Accepts heterogeneous inputs.

```
const ab = new ArrayBuffer(4);
const u8 = new Uint8Array([1, 2, 3, 4]);
const dv = new DataView(new ArrayBuffer(2));

const result = ArrayBuffer.concat([ab, u8, dv]);
// result.byteLength === 10
```

Options:

- `length` — control result byte length (truncate or zero-fill)
- `resizable: true` — result is resizable; `length` becomes `maxByteLength`
- `immutable: true` — result is immutable (depends on Immutable ArrayBuffer proposal)
- `resizable` and `immutable` are mutually exclusive

`SharedArrayBuffer.concat(items [, options])`

Byte-level concatenation into a new SharedArrayBuffer. Same heterogeneous inputs.

```
const sab = new SharedArrayBuffer(4);
const u8 = new Uint8Array([1, 2, 3, 4]);

const result = SharedArrayBuffer.concat([sab, u8]);
// result.byteLength === 8
```

Options:

- `length` — control result byte length (truncate or zero-fill)
- `growable: true` — result is growable; `length` becomes `maxByteLength`
- No `immutable` option (SharedArrayBuffer does not support immutability)

Spec Text Status

Full spec text is available at the proposal repository.

Spec defines:

- `%TypedArray%.concat` — element-oriented with `CopyDataBlockBytes`
- `ArrayBuffer.concat` — byte-oriented with options bag
- `SharedArrayBuffer.concat` — byte-oriented with options bag
- `GetConcatenationSources` — shared abstract operation for buffer methods
- `CopySourcesToBuffer` — shared abstract operation for the copy loop
- `ValidateIntegralNumber` — validates the length argument

Editor's notes explicitly allow implementation-defined optimization:

> *"Implementations may use any technique – such as bulk memory copy, deferred allocation, or platform-specific optimizations – provided the observable result is equivalent."*

Design Decisions Since Stage 1

Feedback from the November 2025 plenary and subsequent issue discussion led to:

1. **Static method** rather than prototype method (`Uint8Array.concat( ... )` not `u8.concat( ... )`)
2. **Iterable argument** rather than rest params (`concat([a, b])` not `concat(a, b)`)
3. **Same-type enforcement** for `%TypedArray%.concat` — mixed types handled at the buffer level
4. **Separate** `ArrayBuffer.concat` / `SharedArrayBuffer.concat` for byte-level heterogeneous concatenation
5. **Options bag** for buffer methods — `length`, `resizable` / `growable`, `immutable`
6. `immutable` **option** added for `ArrayBuffer.concat` (depends on Immutable ArrayBuffer, stage 2.7)
7. **Iterable fully consumed** before validation (current spec text uses `IteratorToList` upfront)

Open Issues

Open: Name Bikeshed

#5 – @mhofman

Is `concat` the right name?

Concerns raised:

- `concat` is currently a **prototype method** on `Array` and `String` — this is a **static method**
- Potential conflict with Node.js `Buffer.concat` if argument shape differs

Alternative suggestions:

- `fromChunks` — avoids overloading `from`, clearly conveys intent
- `join` — but conflicts with iterator helpers and `Array.prototype.join`

Current spec text uses `concat`. Seeking committee input on naming.

Open: Iterable Consumption Ordering

#7 – @bakkot

Should the iterable be fully consumed before validating items?

Two approaches:

	Consume first, validate after	Validate as you go
Pros	No user code between length computation and copy; optimizable	Fails fast on invalid input
Cons	Consumes entire iterable even if first item is invalid	Iterator could detach/resize earlier items mid-iteration

Current spec text: **consumes first** (`IteratorToList` then validate).

@bakkot weakly prefers consuming upfront (optimize for the happy path).

Open: Accept Iterables as Items?

#8 – @jasnell

Should `%TypedArray%.concat` accept plain iterables alongside TypedArrays?

```
// Currently: only TypedArrays
Uint8Array.concat([new Uint8Array([1, 2]), new Uint8Array([3, 4])]);

// Proposed extension: also accept iterables?
Uint8Array.concat([new Uint8Array([1, 2]), [3, 4]]);
```

Considerations:

- Convenience for mixed sources (TypedArray + plain array)
- Complicates length pre-computation (iterables have unknown length)
- May undermine the optimization story (can't `CopyDataBlockBytes` from a plain array)
- `%TypedArray%.from` already handles the iterable-to-TypedArray case

Seeking committee guidance. Current spec only accepts TypedArrays.

Open: Immutable ArrayBuffer Integration

#4 – @mhofman

How should this proposal integrate with the Immutable ArrayBuffer proposal?

Current approach: `ArrayBuffer.concat([ ... ], { immutable: true })`

- Avoids the round-trip of creating a mutable buffer then calling `transferToImmutable`
- `MakeArrayBufferImmutable` is a placeholder AO pending the Immutable AB proposal (stage 2.7)
- `resizable` and `immutable` are mutually exclusive

Open sub-questions:

- Should all-immutable inputs automatically produce an immutable result? (Likely no — brittle, refactoring hazard)
- Is the options bag the right mechanism? (@mhofman says yes)

Current spec includes the option. Concrete semantics depend on Immutable AB landing.

Other Open Issues

ArrayBuffer concat semantics (#2) – @bengl

- What does the backing buffer look like for partial-view TypedArrays?
- **Addressed in spec:** only the viewed portion is included; result buffer is exactly the concatenated bytes

Mixed types (#3) – @bengl

- Should mixed TypedArray types be supported?
- **Addressed in spec:** `%TypedArray%.concat` requires same-type; `ArrayBuffer.concat` / `SharedArrayBuffer.concat` accept heterogeneous inputs at the byte level

Summary

Issue	Status	Seeking
Name: `concat` vs `fromChunks` vs other	Open	Committee preference
Iterable consumption ordering	Open	Committee confirmation
Accept plain iterables as items	Open	Committee guidance
Immutable ArrayBuffer integration	In spec (placeholder)	Dependent on Immutable AB
ArrayBuffer concat semantics	Addressed in spec	–
Mixed types	Addressed in spec	–

Asking for Stage 2

Stage 2 entrance criteria:

- Initial spec text covering all major semantics
- Committee expectation that the feature will be developed and eventually included
- Identified open issues are appropriate for Stage 2 resolution

What we have:

- Complete spec text for all three methods
- Shared abstract operations (`GetConcatenationSources` , `CopySourcesToBuffer`)
- Explicit affordance for implementation optimization
- Clear set of open questions suitable for stage 2 refinement

Requesting advancement to Stage 2.